Loan Prediction

Introduction Loan Prediction Problem

Problem Statement

- Assume XYZ company deals in all home loans. They have a presence across all urban, semi-urban and rural areas. Customers first apply for a home loan after that company validates the customer's eligibility for a loan. The company wants to automate the loan eligibility process (real-time) based on customer detail provided while filling out the online application form.
- These details are Gender, Marital Status, Education, Number of Dependents, Income, Loan Amount, Credit History, and others. To automate this process, they have given a problem to identify the customer segments, that are eligible for loan amounts so that they can specifically target these customers.

It is a classification problem where we have to predict whether a loan would be approved or not. In these kinds of problems, we have to predict discrete values based on a given set of independent variables (s). Classification can be of two types:

- **Binary Classification**:- In this, we have to predict either of the two given classes. For example: classifying the "gender" as male or female, predicting the "result" as to win or loss, etc.
- **MultiClass Classification**:- Here we have to classify the data into three or more classes. For example: classifying a "movie's genre" as comedy, action, or romantic, classifying "fruits" like oranges, apples, pears, etc.

Loan prediction is a very common real-life problem that each retail bank faces at least once in its lifetime. If done correctly, it can save a lot of man-hours at the end of a retail bank.

Hypothesis generation

Now that we have confirmed the problem, let's move on to hypothesis generation. This is the process of listing all the factors that can affect your results.

What is Hypothesis Generation?

This is a very important step in the data science/machine learning pipeline. It's about understanding the problem in detail by brainstorming about the greatest potential that can affect your results. This is done by a complete understanding of the problem before reviewing the data.

- Salary: Applicants with high income should have more chances of getting approval.
- Previous history: Applicants who have paid their historical debts have more chances of getting approval.
- Loan amount: Less the amount higher the chances of getting approval.
- Loan term: Less the time period has higher chances of approval.
- EMI: Lesser the amount to be paid monthly, the higher the chances of getting approval.

These are some of the factors which we think can affect Loan Approval, the dependent variable for this loan prediction problem.

Loading the Data

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
import warnings
warnings.filterwarnings("ignore")
```

Data

For this problem, we have been given three CSV files: train, test, and sample submission.

- Train file will be used for training the model, i.e. our model will learn from this file. It contains all the independent variables and the target variable.
- The test file contains all the independent variables, but not the target variable. We will apply the model to predict the target variable for the test data.
- The sample/Output submission file contains the format in which we have to submit our predictions.

Reading Data

Importing Data into Python

```
train = pd.read_csv('train.csv')
train.head()
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Pro
0	LP001002	Male	No	0	Graduate	No	5849	0.0	NaN	360.0	1.0	
1	LP001003	Male	Yes	1	Graduate	No	4583	1508.0	128.0	360.0	1.0	
2	LP001005	Male	Yes	0	Graduate	Yes	3000	0.0	66.0	360.0	1.0	
3	LP001006	Male	Yes	0	Not Graduate	No	2583	2358.0	120.0	360.0	1.0	
4	LP001008	Male	No	0	Graduate	No	6000	0.0	141.0	360.0	1.0	

```
test = pd.read_csv('test.csv')
test.head()
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Pro
0	LP001015	Male	Yes	0	Graduate	No	5720	0	110.0	360.0	1.0	
1	LP001022	Male	Yes	1	Graduate	No	3076	1500	126.0	360.0	1.0	
2	LP001031	Male	Yes	2	Graduate	No	5000	1800	208.0	360.0	1.0	
3	LP001035	Male	Yes	2	Graduate	No	2340	2546	100.0	360.0	NaN	
4	LP001051	Male	No	0	Not Graduate	No	3276	0	78.0	360.0	1.0	

Understanding the Data

```
train.columns
```

```
Index(['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education',
       'Self_Employed', 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount',
       'Loan_Amount_Term', 'Credit_History', 'Property_Area', 'Loan_Status'],
      dtype='object')
```

- From the above we can say that, there are 12 independent variables and 1 target variable, i.e. Loan_Status in the train dataset

```
test.columns
```

```
Index(['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education',
       'Self_Employed', 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount',
       'Loan_Amount_Term', 'Credit_History', 'Property_Area'],
      dtype='object')
```

- We have similar features in the test dataset as the train dataset except the Loan_Status. We will predict the Loan_Status using the model built using the train data

```
train.dtypes
```

```
Loan_ID               object
Gender                object
Married               object
Dependents            object
Education             object
Self_Employed         object
ApplicantIncome        int64
CoapplicantIncome    float64
LoanAmount           float64
```

- We have 614 rows and 13 columns in the training dataset and 367 rows and 12 columns in the test dataset.

Univariate Analysis

We will do a univariate analysis, analyzing data where we analyze each variable individually. For categorical features, we can use frequency tables or bar plots to calculate the number of each category in a particular variable. Probability Density Functions(PDF) can be used to look at the distribution of the numerical variables.

Target Variable

We will first look at the target variable, i.e., Loan_Status. As it is a categorical variable, let us look at its frequency table, percentage distribution, and bar plot.

The frequency table of a variable will give us the count of each category in that variable.

```
train['Loan_Status'].value_counts()
```

```
Y    422
N    192
Name: Loan_Status, dtype: int64
```

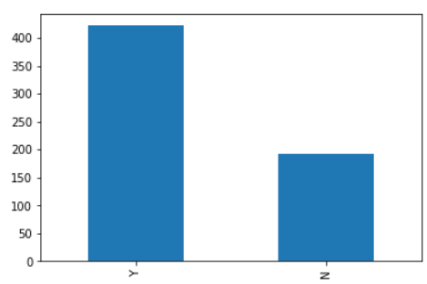
- Normalize can be to true to print proportions instrad of number

```
train['Loan_Status'].value_counts(normalize=True)
```

```
Y    0.687296
N    0.312704
Name: Loan_Status, dtype: float64
```

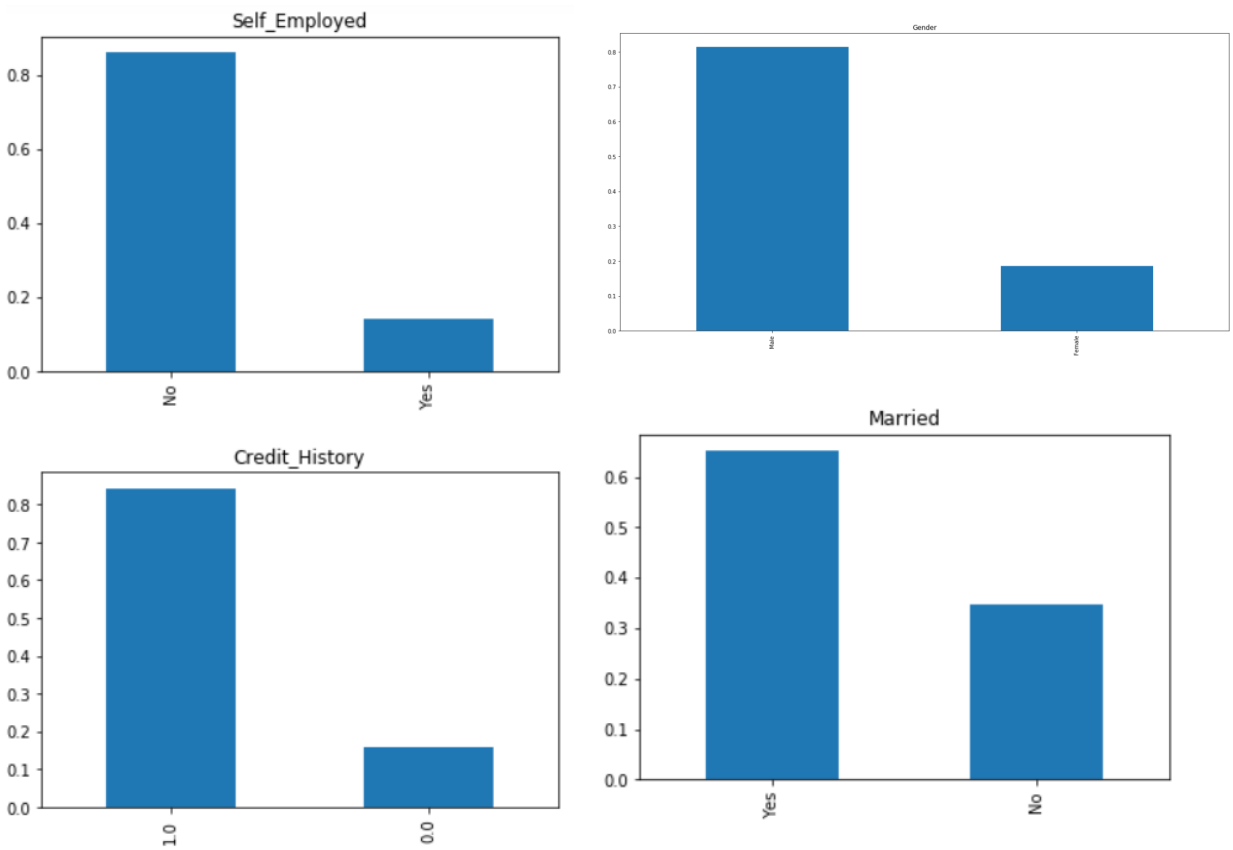
```
train['Loan_Status'].value_counts().plot.bar()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x2cb78b71488>
```



The loan of 422(around 70%) people out of 614 were approved.

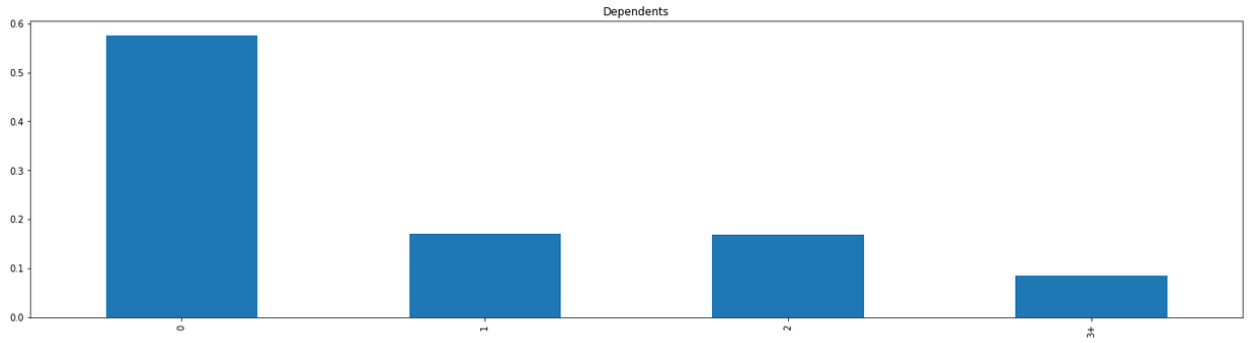
Independent Variable (Categorical)

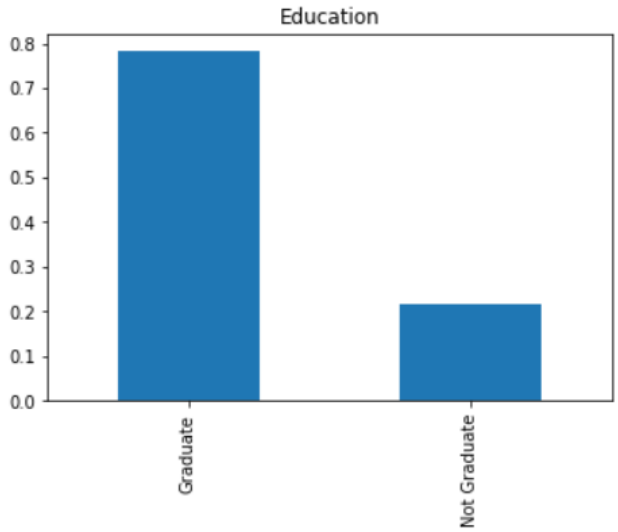


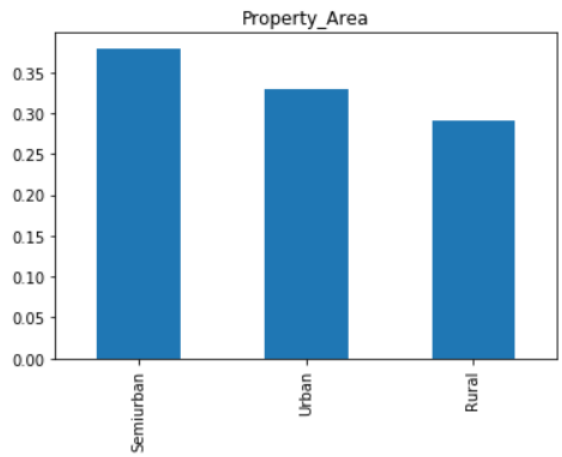
It can be inferred from the above bar plots that:

- Male = 80%
- Married = 65%
- Self-employed 15%
- 85% of applicants have repaid their debts

Independent Variable (Ordinal)



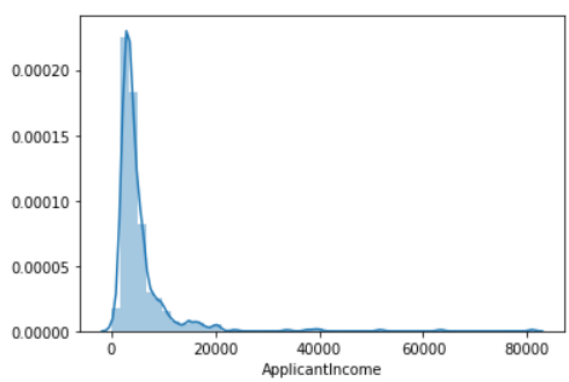




Inferences can be made from the above bar plots:

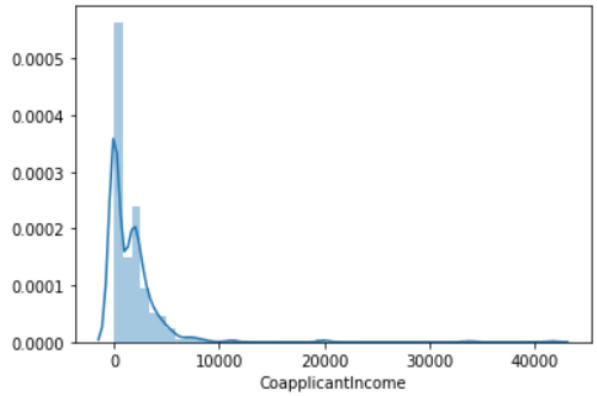
- Most of the applicants don't have dependents.
- About 80% of the applicants are graduates.
- Most of the applicants are from semi-urban areas

Independent Variable (Numerical)



From this, we can conclude that most of the applicant's income distribution data is to the left, that is, not normally distributed. The algorithm works well if the data is normally distributed, so we will try to make it normally distributed.

Let's check coapplicantincome



Bivariate Analysis

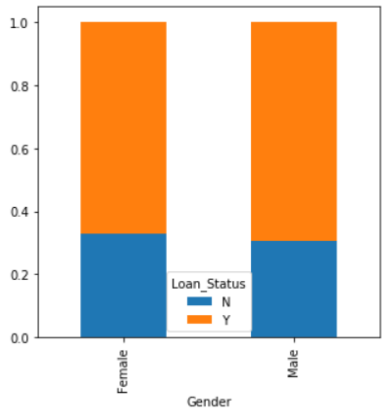
Let’s recall some of the hypotheses that we generated earlier:

- Applicants with high incomes should have more chances of loan approval.
- Applicants who have repaid their previous debts should have higher chances of loan approval.
- Loan approval should also depend on the loan amount. If the loan amount is less, the chances of loan approval should be high.
- Lesser the amount to be paid monthly to repay the loan, the higher the chances of loan approval.

Let’s try to test the above-mentioned hypotheses using bivariate analysis

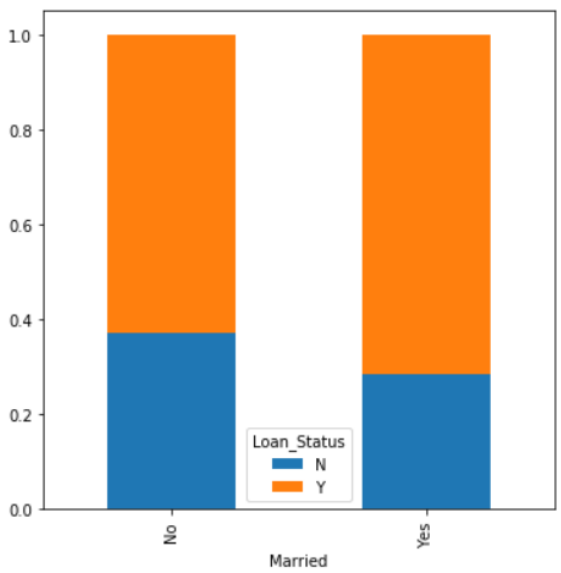
Categorical Independent Variable vs Target Variable

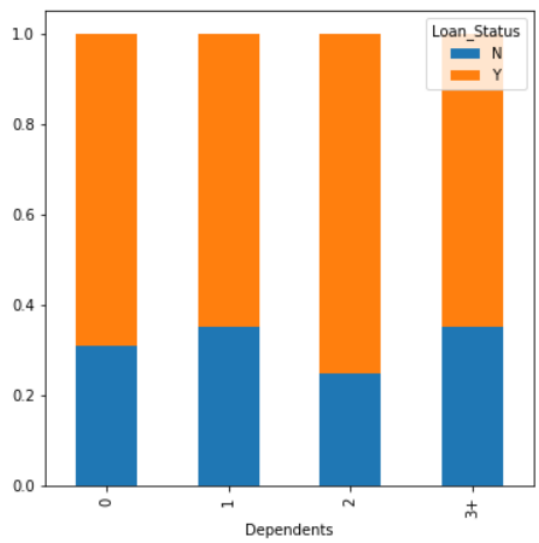
The proportion of approved and unapproved loans

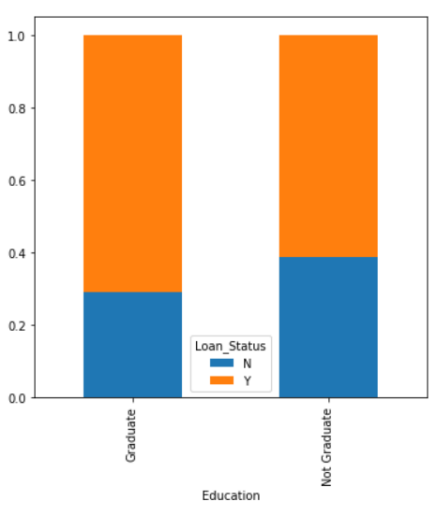


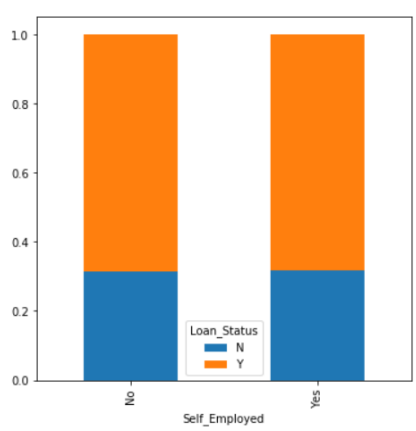
- It can be inferred that the proportion of male and female applicants is more or less the same for both approved and unapproved loans.

Categorical Variables vs the Target Variables



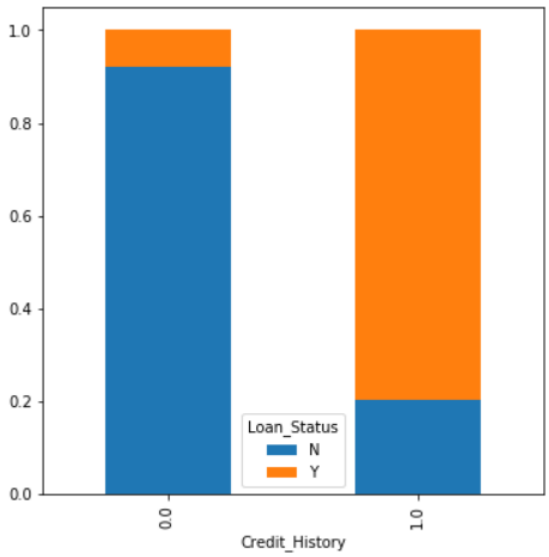


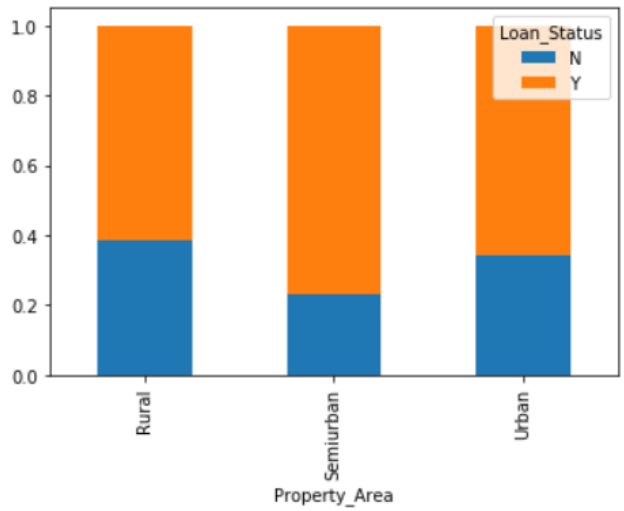




- The proportion of married applicants is higher for the approved loans.
- The distribution of applicants with 1 or 3+ dependents is similar across both the categories of Loan_Status.
- There is nothing significant we can infer from Self_Employed vs Loan_Status plot.

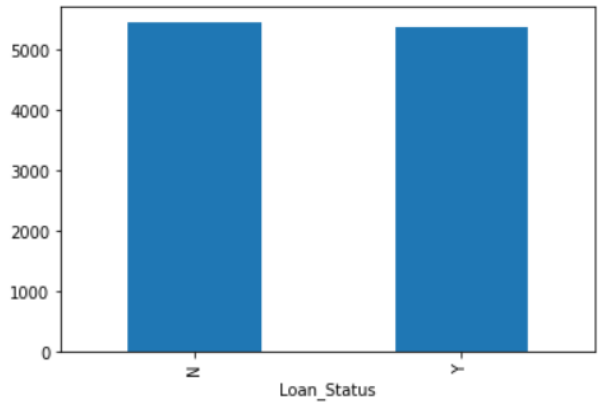
Categorical independent variables and Loan_Status



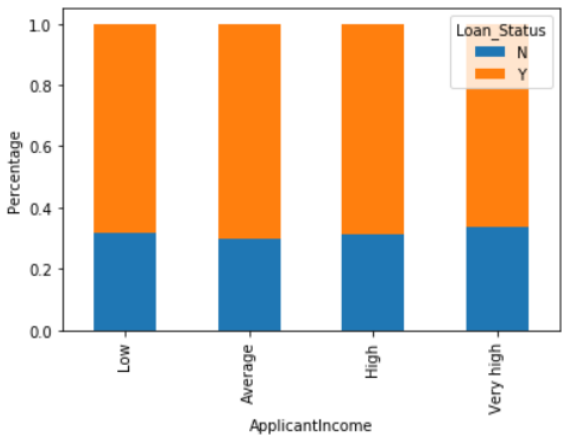


- It seems people with a credit history of 1 are more likely to get their loans approved.
- The proportion of loans getting approved in semi-urban areas is higher as compared to that in rural or urban areas.

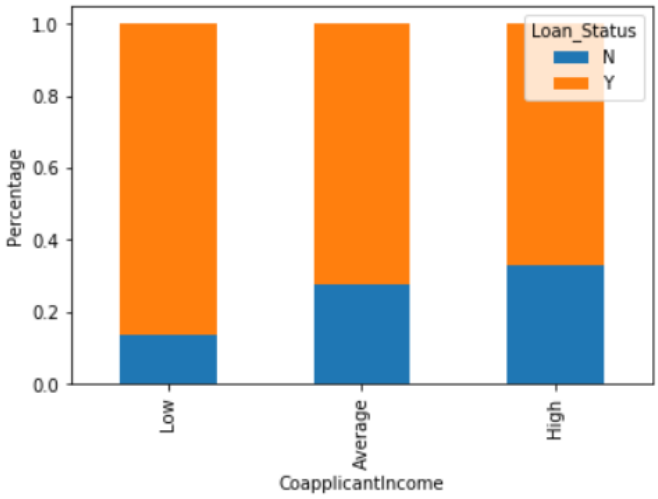
Numerical Independent Variable vs Target Variable



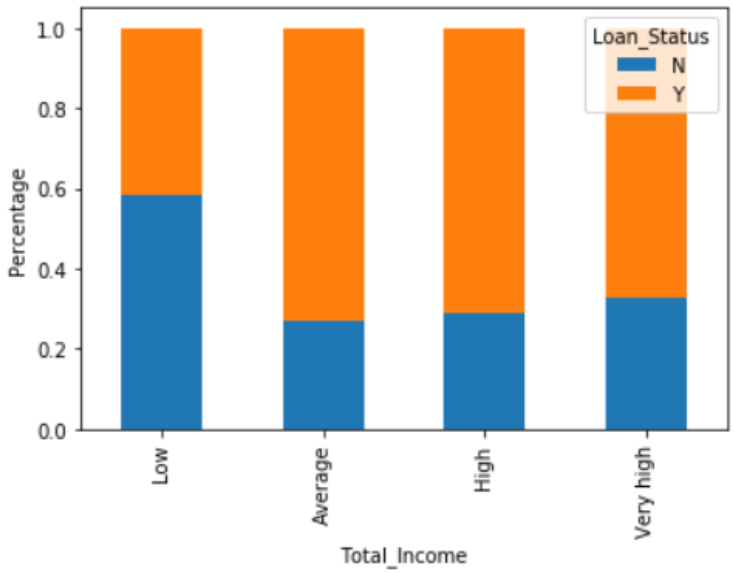
Here the y-axis represents the mean applicant income. We don't see any change in the mean income. So, let's make bins for the applicant income variable based on the values in it and analyze the corresponding loan status for each bin.



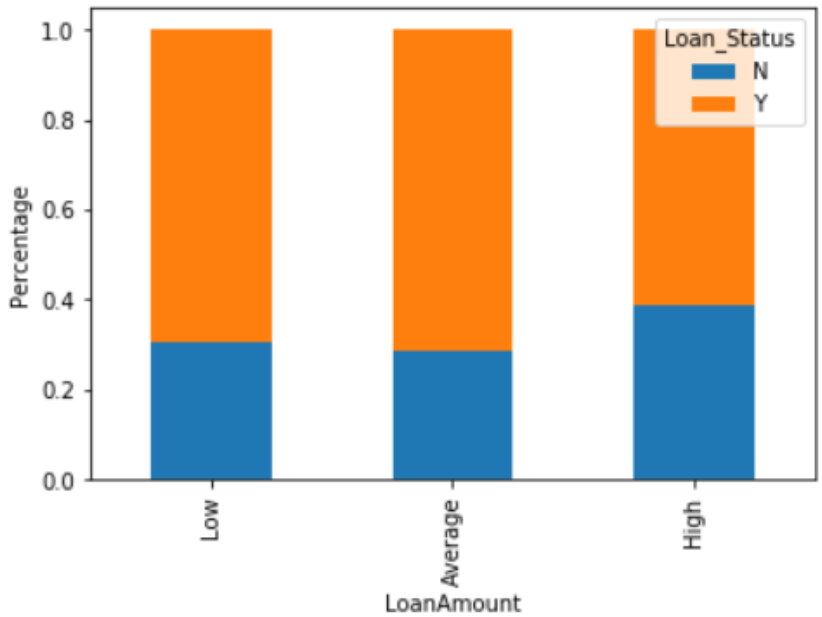
It can be inferred that Applicant's income does not affect the chances of loan approval which contradicts our hypothesis in which we assumed that if the applicant's income is high the chances of loan approval will also be high.



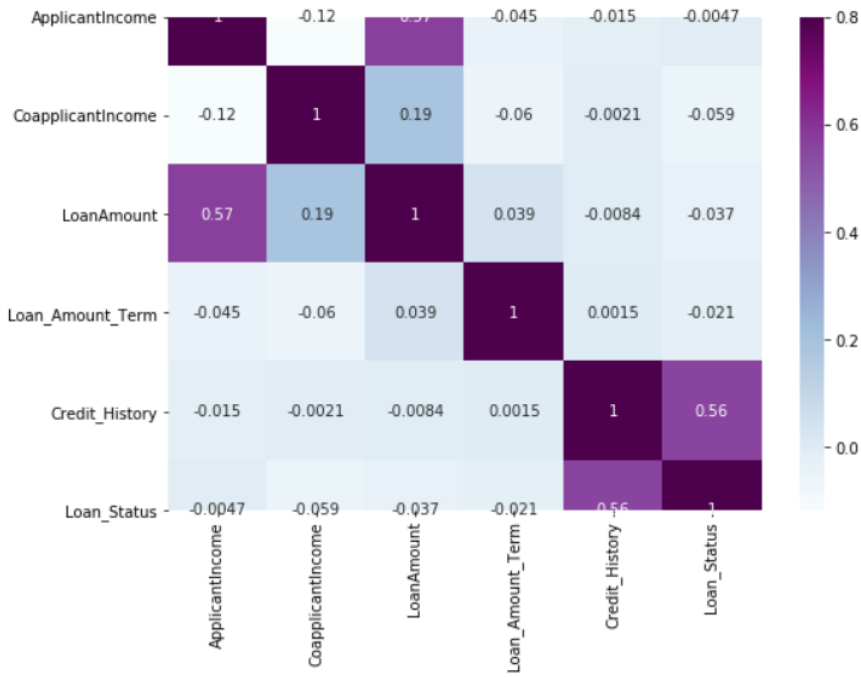
It shows that if co-applicants income is less the chances of loan approval are high. But this does not look right. The possible reason behind this may be that most of the applicants don't have any co-applicant so the co-applicant income for such applicants is 0 and hence the loan approval is not dependent on it. So we can make a new variable in which we will combine the applicant's and co-applicants' income to visualize the combined effect of income on loan approval.



We can see that Proportion of loans getting approved for applicants having low Total_Income is very less as compared to that of applicants with Average, High, and Very High Income.



It can be seen that the proportion of approved loans is higher for Low and Average Loan Amounts as compared to that of High Loan Amounts which supports our hypothesis which considered that the chances of loan approval will be high when the loan amount is less.



We see that the most correlated variables are (ApplicantIncome – LoanAmount) and (Credit_History – Loan_Status). LoanAmount is also correlated with CoapplicantIncome.

Missing Value and Outlier Treatment

```
train.isnull().sum()
```

```
Loan_ID              0
Gender              13
Married              3
Dependents          15
Education            0
Self_Employed       32
ApplicantIncome      0
CoapplicantIncome    0
LoanAmount          22
Loan_Amount_Term    14
Credit_History      50
Property_Area        0
Loan_Status          0
dtype: int64
```

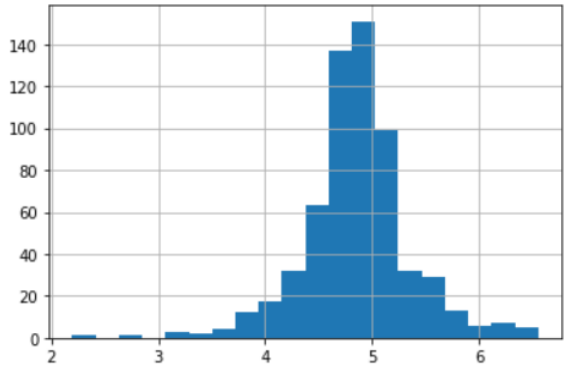
```
train['LoanAmount'].fillna(train['LoanAmount'].median(), inplace=True)
```

```
train.isnull().sum()
```

```
Loan_ID              0
Gender               0
Married              0
Dependents           0
Education            0
Self_Employed        0
ApplicantIncome      0
CoapplicantIncome    0
LoanAmount           0
Loan_Amount_Term     0
Credit_History       0
Property_Area        0
Loan_Status          0
dtype: int64
```

Outlier Treatment

```
train['LoanAmount_log']=np.log(train['LoanAmount'])
train['LoanAmount_log'].hist(bins=20)
test['LoanAmount_log']=np.log(test['LoanAmount'])
```



The distribution looks much closer to normal and the effect of extreme values has significantly subsided. Let's build a logistic regression model and make predictions for the test dataset.

Precision: It is a measure of correctness achieved in true prediction i.e. of observations labeled as true, how many are actually labeled true

Precision = TP / (TP + FP)

Recall(Sensitivity): It is a measure of actual observations which are predicted correctly i.e. how many observations of true class are labeled correctly. It is also known as 'Sensitivity'.

Recall = TP / (TP + FN)

Specificity: It is a measure of how many observations of false class are labeled correctly.

Specificity = TN / (TN + FP)

Specificity and Sensitivity play a crucial role in deriving the ROC curve.

ROC curve

Receiver Operating Characteristic(ROC) summarizes the model's performance by evaluating the trade-offs between true positive rate (sensitivity) and false-positive rate(1- specificity).

The area under the curve (AUC), referred to as an index of accuracy(A) or concordance index, is a perfect performance metric for the ROC curve. Higher the area under the curve, the better the prediction power of the model.

The area of this curve measures the ability of the model to correctly classify true positives and true negatives. We want our model to predict the true classes as true and false classes as false.

Model Building: Part 1

Our first model predicts the target variable. Starting with Logistic Regression which is used for predicting binary outcomes.

- Logistic Regression is a classification algorithm. It is used to predict a binary outcome (1 / 0, Yes / No, True / False) given a set of independent variables.
- Logistic regression is an estimation of the Logit function. The logit function is simply a log of odds in favor of the event.
- We will use "scikit-learn" (sklearn) for making different models which is an open-source library for Python. It is one of the most efficient tools which contains many inbuilt functions that can be used for modeling in Python.
- Sklearn requires the target variable in a separate dataset. So, we will drop our target variable from the training dataset and save it in another dataset.

Now we will make dummy variables for the categorical variables. A dummy variable turns categorical variables into a series of 0 and 1, making them a lot easier to quantify and compare. Let us understand the process of dummies first:

- Consider the "Gender" variable. It has two classes, Male and Female.
- As logistic regression takes only the numerical values as input, we have to change male and female into a numerical value.
- Once we apply dummies to this variable, it will convert the "Gender" variable into two variables(Gender_Male and Gender_Female), one for each class, i.e. Male and Female.
- Gender_Male will have a value of 0 if the gender is Female and a value of 1 if the gender is Male.

```
X = pd.get_dummies(X)
train=pd.get_dummies(train)
test=pd.get_dummies(test)
```

```
from sklearn.model_selection import train_test_split
x_train, x_cv, y_train, y_cv = train_test_split(X,y, test_size=0.3)
```

- The dataset has been divided into training and validation part. Let us import LogisticRegression and accuracy_score from sklearn and fit the logistic regression model.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
model = LogisticRegression()
model.fit(x_train, y_train)
```

```
LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,
                   intercept_scaling=1, l1_ratio=None, max_iter=100,
                   multi_class='warn', n_jobs=None, penalty='l2',
                   random_state=None, solver='warn', tol=0.0001, verbose=0,
                   warm_start=False)
```

```
pred_cv = model.predict(x_cv)
accuracy_score(y_cv,pred_cv)
```

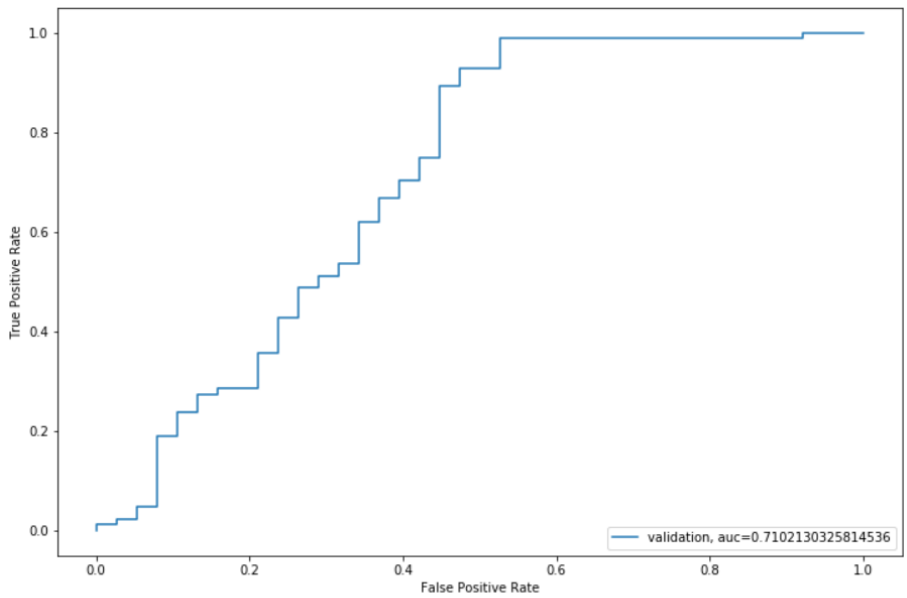
```
0.7945945945945946
```

So our predictions are almost 80% accurate, i.e. we have identified 80% of the loan status correctly

```
i=1
mean = 0
kf = StratifiedKFold(n_splits=5,random_state=1)
for train_index,test_index in kf.split(X,y):
    print ('\n{} of kfold {} '.format(i,kf.n_splits))
    xtr,xvl = X.loc[train_index],X.loc[test_index]
    ytr,yvl = y[train_index],y[test_index]
    model = LogisticRegression(random_state=1)
    model.fit(xtr,ytr)
    pred_test=model.predict(xvl)
    score=accuracy_score(yvl,pred_test)
    mean += score
    print ('accuracy_score',score)
    i+=1
    pred_test = model.predict(test)
    pred = model.predict_proba(xvl)[:,1]
print ('\n Mean Validation Accuracy',mean/(i-1))
```

```
1 of kfold 5
accuracy_score 0.8145161290322581

2 of kfold 5
accuracy_score 0.7580645161290323

3 of kfold 5
accuracy_score 0.7868852459016393

4 of kfold 5
accuracy_score 0.860655737704918

5 of kfold 5
accuracy_score 0.819672131147541

 Mean Validation Accuracy 0.8079587519830778
```

- Instead of creating a validation set, we can also make use of cross-validation to validate our predictions.
- We got an accuracy of 0.8079 on the leaderboard
- So our predictions are almost 80% accurate, i.e. we have identified 80% of the loan status correctly.



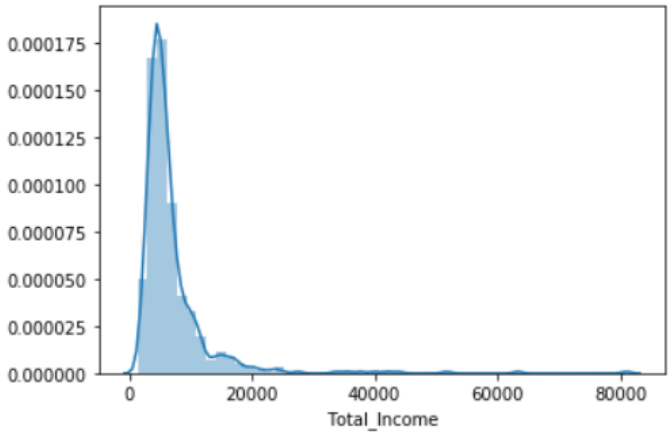
- We got an auc value of 0.71

We got an accuracy of 0.71021 on the leaderboard. Now we will try to improve this accuracy using different approaches.

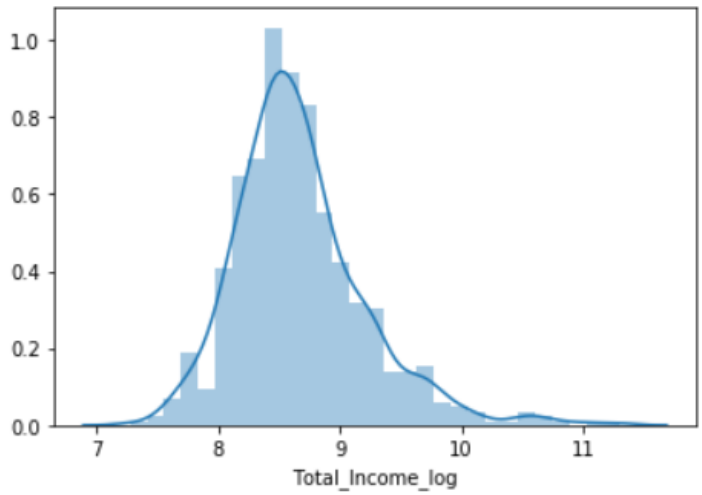
Feature Engineering

Based on the domain knowledge, we can come up with new features that might affect the target variable. We will create the following three new features:

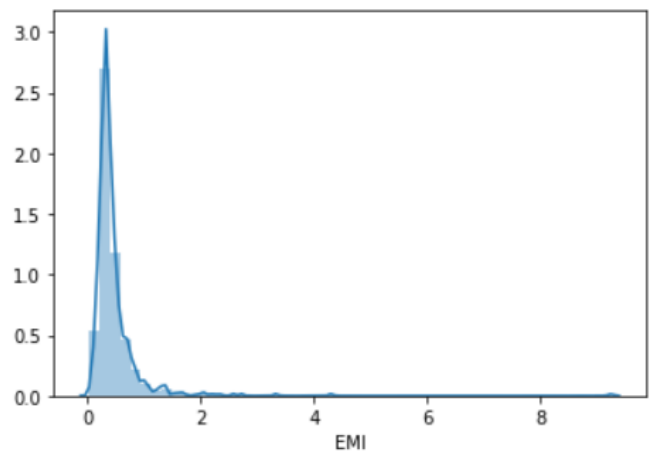
- Total Income:- As discussed during bivariate analysis we will combine the Applicant Income and Co-applicant Income. If the total income is high, the chances of loan approval might also be high.
- EMI:- EMI is the monthly amount to be paid by the applicant to repay the loan. The idea behind making this variable is that people who have high EMI's might find it difficult to pay back the loan. We can calculate the EMI by taking the ratio of loan amount with respect to loan amount term.
- Balance Income:- This is the income left after the EMI has been paid. The idea behind creating this variable is that if this value is high, the chances are high that a person will repay the loan and hence increasing the chances of loan approval.



- It is shifted towards the left, i.e., the distribution is right-skewed. So, let's take the log transformation to make the distribution normal.



- Now the distribution looks much closer to normal and the effect of extreme values has been significantly subsided. Let's create the EMI feature now.



- Balance Income feature now and check its distribution.

Model Building: Part 2

Random Forest

- Similarly building a model by using Random Forest

```
i=1
mean = 0
kf = StratifiedKFold(n_splits=5,random_state=1,shuffle=True)
for train_index,test_index in kf.split(X,y):
    print ('\n{} of kfold {} '.format(i,kf.n_splits))
    xtr,xvl = X.loc[train_index],X.loc[test_index]
    ytr,yvl = y[train_index],y[test_index]
    model = RandomForestClassifier(random_state=1, max_depth=10)
    model.fit(xtr,ytr)
    pred_test=model.predict(xvl)
    score=accuracy_score(yvl,pred_test)
    mean += score
    print ('accuracy_score',score)
    i+=1
    pred_test = model.predict(test)
    pred = model.predict_proba(xvl)[:,1]
print ('\n Mean Validation Accuracy',mean/(i-1))
```

```
1 of kfold 5
accuracy_score 0.7338709677419355

2 of kfold 5
accuracy_score 0.7903225806451613

3 of kfold 5
accuracy_score 0.7868852459016393

4 of kfold 5
accuracy_score 0.7704918032786885

5 of kfold 5
accuracy_score 0.8524590163934426

 Mean Validation Accuracy 0.7868059227921735
```

- Mean validation accuracy for this model is 0.786

Conclusion

There are still quite a many things that can be tried to improve our models' predictions. We create and add more variables, try different models with a different subset of features and/or rows, etc. Some of the ideas are listed below

- We can combine the applicants with dependents and make a new feature
- We can also make independent vs independent variable visualizations to discover some more patterns.
- We can also arrive at the EMI using a better formula which may include interest rates as well.
- We can even try ensemble modeling (a combination of different models).

Hence :

- Logistic Regression Model Gives us Mean Validation Accuracy of = 80% ¶

- Random Forest Model Gives us Mean Validation Accuracy of = 78%

Logistic Regression Model is Better for this dataset